

ScamGuard

A Machine Learning-Powered Proactive Defense Against
Employment Fraud

Course Name: CS 69099-001: Computer Science Capstone

Team: **Echo**

Team Members:

Venkat Sai Emani

Mounika Seelam

Akheela Dusa

Khyathiswaroop Pathangay

Nikhitha Neerugatti

Veera Reddy

Rongan Bai

Date of Submission: December 10, 2025

Executive Summary

The job search process has increasingly become a vector for financial fraud and identity theft. In 2024 alone, job-related scams cost victims over \$350 million, disproportionately affecting vulnerable populations such as international students, recent graduates, and immigrants who are unfamiliar with local hiring norms. Existing solutions, such as manual moderation on job boards, are fundamentally reactive. They typically remove a fraudulent posting only after a user has reported it or fallen victim to it.

We have developed ScamGuard to close this security gap. It is a machine learning-powered web application designed to provide proactive, real-time fraud detection. Rather than relying on blacklists or user reports, ScamGuard analyzes the linguistic patterns and metadata of a job description instantly. By determining the probability of fraud before a user applies, the system serves as a preventative shield, empowering job seekers to make informed decisions and protecting their personal and financial data.

Goals and Objectives: The primary objective of our team was to build a system capable of distinguishing between legitimate and fraudulent job postings with a high degree of reliability. Our specific goals included:

- **Safety-First Detection:** Prioritizing Recall over simple accuracy. Our goal was to minimize "False Negatives" (missed scams) to ensure that dangerous listings are rarely marked as safe.
- **Hybrid Intelligence:** Implementing a feature extraction pipeline that combines Natural Language Processing (NLP) to analyze unstructured text with Metadata Heuristics to flag structural warning signs (e.g., missing company logos or suspicious email domains).
- **Accessibility:** Delivering the model via a lightweight, user-friendly Web UI that requires no technical expertise to operate.
- **Production Deployment:** Successfully deploying the application to the university's Linux server (Cassini) with a robust CI/CD pipeline.

Major Deliverables and Features:

The project resulted in a fully functional, end-to-end software solution comprised of three core subsystems:

1. **The Backend Engine:** A Python-based backend utilizing a Random Forest Classifier. The model is trained on a curated dataset of over 29,000 job postings, utilizing TF-IDF vectorization to transform raw text into 5,000 numerical features.
2. **The ETL Pipeline:** A robust data ingestion system that cleanses HTML tags, removes stopwords, and handles data corruption (fault-tolerant parsing) to prepare raw data for inference.
3. **The User Interface:** A Flask-based web application deployed on the Cassini server (<http://cassini.cs.kent.edu:9620/>). Users can paste job descriptions to receive an instant "Risk Score" (0-100%) and a classification label (Likely Legit, Needs Review, or High Risk).

Accomplishments and Results: The project was successful, meeting all core technical requirements.

- **Model Performance:** After comparing Logistic Regression and Random Forest models, we selected the Random Forest approach. It achieved an overall Accuracy of 98.6% and, most critically, a Recall of 97%. This means the system successfully identifies 97 out of every 100 scams it encounters.
- **Data Insights:** Our Exploratory Data Analysis (EDA) revealed critical behavioral patterns of scammers, such as the high correlation between fraudulent posts and the absence of a company logo. We also disproved our initial hypothesis regarding remote work, statistical analysis

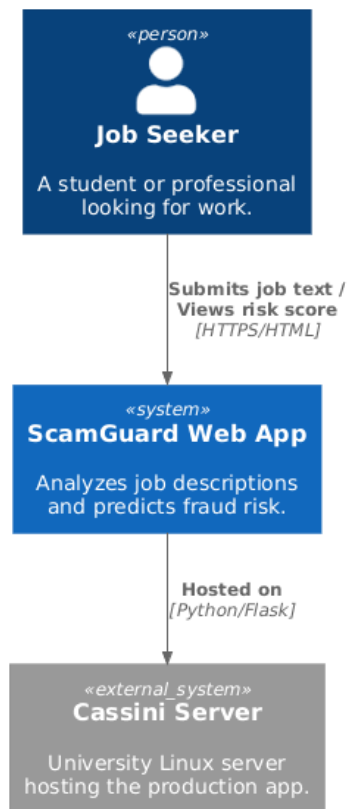
revealed that in our dataset, jobs explicitly marked as "Remote" were safer than those that were not, allowing us to use this as a positive predictor.

- **Infrastructure:** We successfully established a professional DevOps workflow using GitHub Actions for continuous deployment and Git Large File Storage (LFS) to manage heavy model artifacts (>100MB), overcoming significant constraints of the hosting environment.

Challenges Overcome:

Our team navigated several technical hurdles during development. We faced severe data integrity issues where raw datasets contained malformed rows ("EOF inside string" errors), which required engineering a custom, fault-tolerant parsing engine. We also resolved deployment bottlenecks related to GitHub's file size limits by migrating to Git LFS. Finally, we addressed the challenge of unstructured text by implementing a rigorous NLP pipeline to standardize chaotic job descriptions into a format suitable for machine learning.

In conclusion, our project ScamGuard has evolved from a theoretical concept into a deployed, high-accuracy tool that effectively addresses the "hidden crisis" of employment fraud.



Problem Statement

The Core Problem: A Reactive Defense in a Proactive Threat Landscape

The job search process, intended to be a transition to new opportunity, has become a significant source of financial and identity risk. Sophisticated online job scams are a global, multi-million-dollar crisis, designed to steal personal data, demand fraudulent fees for 'training' or 'equipment,' and exploit urgency and anxiety in job seekers.

The fundamental issue is that existing fraud detection methods are reactive. Major job platforms rely heavily on user reports, which means a scam must victimize someone before it is taken down. This approach is inherently insufficient against highly scalable and rapidly evolving fraud operations.

Why this problem is important

The crisis is characterized by staggering financial impact and targeted vulnerability:

- **Financial Impact:** In 2024 alone, job-related scams cost victims over \$350 million. This is not a marginal problem, which is a hidden financial threat.
- **Vulnerability:** Scammers disproportionately target the most vulnerable demographics:
 - Students and Recent Graduates seeking 'Entry-Level' or 'Part-Time' work, who may lack experience in identifying red flags.
 - International Job Seekers who may be unfamiliar with local hiring practices and feel pressured by visa or financial deadlines.

ScamGuard addresses the need to shift the defense mechanism from reactive cleanup to proactive, real-time risk assessment, putting the power to verify safety directly into the user's hands.

Stakeholder Group	What is the need	Benefit from ScamGuard
Vulnerable Job Seekers	The need for security and confidence during the application process, avoiding financial loss.	Real-Time Risk Score: Instant, objective guidance on whether a posting is safe to proceed with.
University Career Center	The goal of protecting student welfare and maintaining institutional reputation.	Tangible Resource: A verifiable, modern tool to recommend to students and recent graduates.

System Overview / Solution Description

ScamGuard is a monolithic, three-tier web application built on a modern Python stack. It is designed for low-latency inference, allowing it to process a job description and return a result in less than one second.

High-Level System Architecture

The system is hosted on the university's Linux server, Cassini (<http://cassini.cs.kent.edu:9620/>), and operates as a service running within a persistent GNU Screen session.

Technology Stack:

- **Frontend:** HTML5, CSS3 (Jinja2 Templates)
- **Backend:** Python 3.9+, Flask
- **Machine Learning:** Scikit-Learn (Random Forest, TF-IDF), NLTK, Pandas, SciPy
- **Deployment:** GitHub Actions, Git LFS, Linux (Cassini), Screen
- **Model Persistence:** Joblib (serialization and loading of trained models .pkl files)

Subsystems and Modules

We have divided the architecture into three functional layers:

- **Frontend Layer:** for user interaction, the *templates/index.html* file renders the user input form and dynamically displays the result from the backend.
- **Backend / Application Layer:** for routing and control, the *app.py* script manages the `/` and `/predict` routes, orchestrating the request from the user to the ML Core.
- **Data Preprocessing Layer:** for feature engineering, the *data_preprocessor.py* module cleans text (strips HTML), calculates linguistic signals (stopword frequency), and extracts key metadata flags (e.g., *has_logo*).

- **ML Core / Inference Engine:** for prediction, the *predictor.py* script loads the serialized Random Forest model, transforms the processed input into a 5,008-feature vector, and calculates the final risk probability score.

Notable Design Decisions and Trade-offs (ADRs)

ADR-001: Model Selection (Random Forest vs. Logistic Regression)

- We needed a model that could handle high-dimensional text data (5,000 features) and capture non-linear relationships between metadata flags and fraud.
- **Trade-off:** Random Forest is more complex and slower to train than Logistic Regression, but it provides superior performance on our critical metric, Recall (97%), ensuring maximal user safety.

ADR-002: Feature Engineering Strategy (TF-IDF + Metadata)

- Context: Text data alone was noisy. Metadata alone was insufficient.
- **Trade-off:** This added complexity to the preprocessing pipeline but was necessary to capture both sophisticated scams (linguistics) and "lazy" scams (missing logo/free email domains) in a single 5,008-feature vector.

ADR-003: Handling Large Model Files via Git LFS

- Context: The trained model and dataset exceeded GitHub's file size limit (>100MB), blocking automated CI/CD deployment.
- **Trade-off:** Implementing LFS added a dependency to the deployment script (`git lfs pull` command). However, it allowed the team to maintain a continuous integration workflow and successfully deploy the entire system to Cassini.

Development Process

Timeline and Milestones The project was executed over a three-month timeline, adhering to the following key phases:

Phase 1: Data Engeering	September 1 – September 30	• Aggregated 4 distinct datasets (29,000+ records). • Built ETL pipeline to resolve schema mismatches. • Performed Data Cleaning (handling <code>ParserError</code>). • Conducted Exploratory Data Analysis (EDA).
Phase 2: Model Development	October 1 to October 31	• Engineered features (TF-IDF + Metadata flags). • Trained baseline Logistic Regression model. • Trained and tuned Random Forest model. • Serialized models using Joblib.

Phase 3: Deployment and Delivery	November 1 to November 30	• Developed Flask Web Application. • Configured GitHub Actions CI/CD & Git LFS. • Deployed to Cassini Production Server. • Final System Validation & Capstone Presentation.
---	---------------------------	--

Teamwork:

We utilized an Agile/Kanban methodology to manage tasks. A GitHub Project Board was established to track tickets across "To Do," "In Progress," and "Done" columns. Weekly stand-up meetings were held to resolve blockers, such as the data corruption issues in Phase 1 and the Git LFS integration in Phase 3.

Roles & Responsibilities:

- **Venkat Sai Emani** (Full Stack): Architected the Flask backend, managed the GitHub repository, and led the integration of the ML model with the web server.
- **Mounika Seelam** (ML Pipeline Architect): Led model selection and training, tuning the Random Forest classifier to maximize Recall.
- **Rongan Bai** (Full Stack): Assisted in API development and ensured the frontend templates rendered dynamic risk scores correctly.
- **Akheela Dusa & Khyathiswaroop Pathangay** (Data Engineering): Built the data ingestion pipeline, resolved CSV parsing errors, and cleaned the dataset for training.
- **Nikhitha Neerugatti & Veera Reddy** (Frontend & Docs): Designed the HTML/CSS interface and compiled technical documentation and presentation materials.

Tools Used

- **Source Control:** GitHub (with Git LFS for binary artifacts).
- **Development:** VS Code, Jupyter Notebooks, Google Colab (for model training).
- **Project Management:** GitHub Projects (Kanban).
- **Communication:** Discord

Implementation Details

Key Components and Logic: The core logic of ScamGuard relies on a Hybrid Feature Extraction strategy, implemented in *data_preprocessor.py* and *predictor.py*.

1. Text Vectorization (The "Brain"):

- We utilize TF-IDF (Term Frequency-Inverse Document Frequency) to convert unstructured job descriptions into a sparse matrix of 5,000 numerical features.
- **Logic:** This allows the model to weigh unique, descriptive terms (e.g., "Python", "SQL") higher than generic stop words, while capturing high-risk vocabulary (e.g., "Urgent", "Wire Transfer").

2. Metadata Heuristics (The "Red Flags"):

- Simultaneously, the system extracts binary flags from the raw input:
 - **has_company_logo:** Checks metadata (simulated in user input via profile completeness).

- **uses_free_email**: Regex check for domains like @gmail.com or @yahoo.com.
- **urgency_flag**: Keyword search for high-pressure terms.

3. Interface Engine:

The RandomForestClassifier takes the combined vector (5,000 text features + 8 metadata features) and outputs a probability score (0.0 to 1.0).

Codebase Structure

Here is the high-level layout of the project root directory: [GitHub](#)

```
ScamGuard/
├── .github/                # GitHub CI/CD workflows
├── static/                 # frontend static assets
├── templates/              # frontend HTML (Jinja2 templates)
│   ├── index.html         # homepage
│   ├── info.html
│   ├── sample.html
│   └── logs.html
├── models/                 # serialized ML model artifacts
│   ├── scamguard_rf_model.pkl
│   ├── scamguard_lr_model.pkl
│   └── tfidf_vectorizer.pkl
├── data/                   # full ML pipeline + training utilities
│   ├── base_model.py      # abstract base class for ML models
│   ├── data_preprocessing.py # text cleaning & preprocessing
│   ├── feature_engineering.py # feature extraction + engineering
│   ├── nlp_pipeline.py    # TF-IDF pipeline + ML inference pipeline
│   ├── text_extractor.py  # extract key fields from job description
│   ├── train_all_models.py # train Logistic Regression, RF, etc.
│   ├── train_xgboost.py   # train XGBoost model
│   ├── engineered_features.csv # engineered dataset for training
│   ├── scamguard.db       # SQLite database (logs & admin data)
│   └── db.py              # database connection handlers
├── api/                    # backend API endpoints
│   ├── __init__.py
│   ├── detect.py          # prediction API endpoint
│   ├── logs.py            # API → fetch logs for admin
│   └── users.py           # signup/Login/Account APIs
├── deployment/             # production deployment configs
│   ├── wsgi.py            # WSGI entrypoint
│   ├── nginx.conf         # Nginx reverse proxy config
│   ├── service.sh         # app start/restart script (if present)
│   └── notes.txt          # deployment instructions
├── account.py              # user account logic
├── auth.py                 # authentication (login/signup)
├── app.py                  # Flask main app + routing
├── gmail_service.py        # gmail API integration
├── init_db.py              # initialize database tables
├── requirements.txt        # python dependencies
├── credentials.json        # google OAuth credentials
└── token.json              # OAuth access/refresh token
```

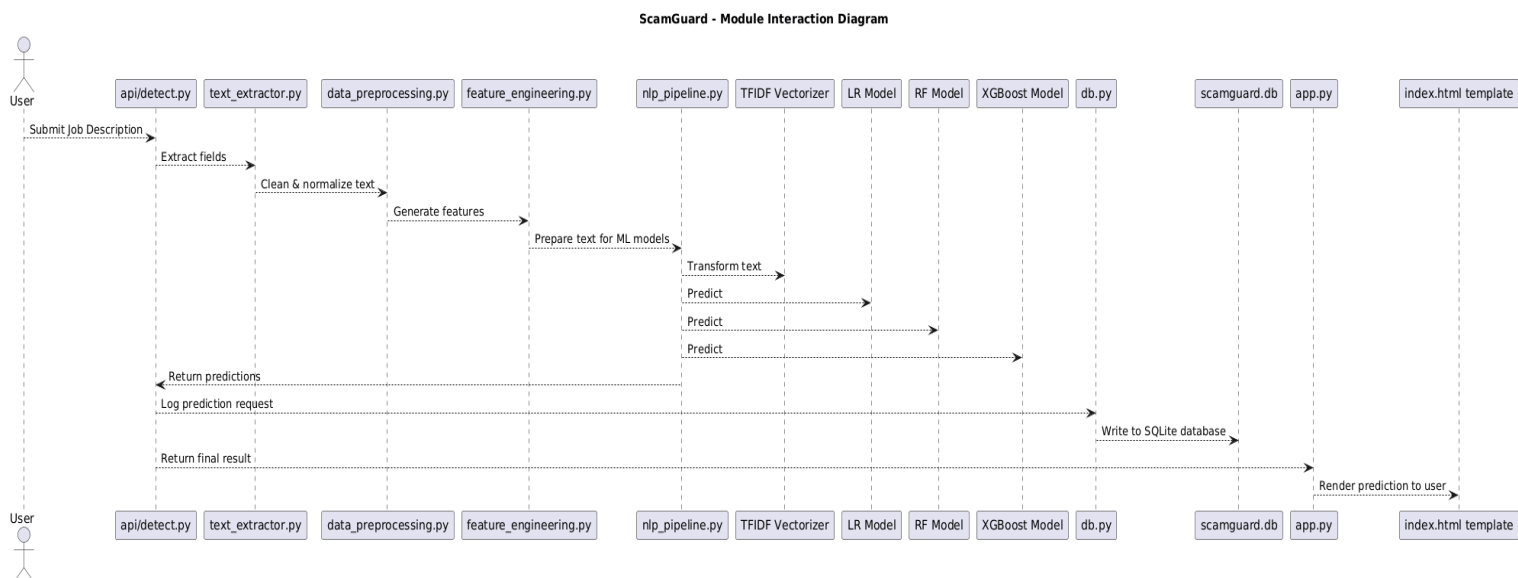
Major Scripts & Modules

The ScamGuard system is built with a modular architecture that separates concerns across frontend, backend, API services, ML pipeline, data handling, and deployment. Below we provided a detailed overview of each major script and its responsibility.

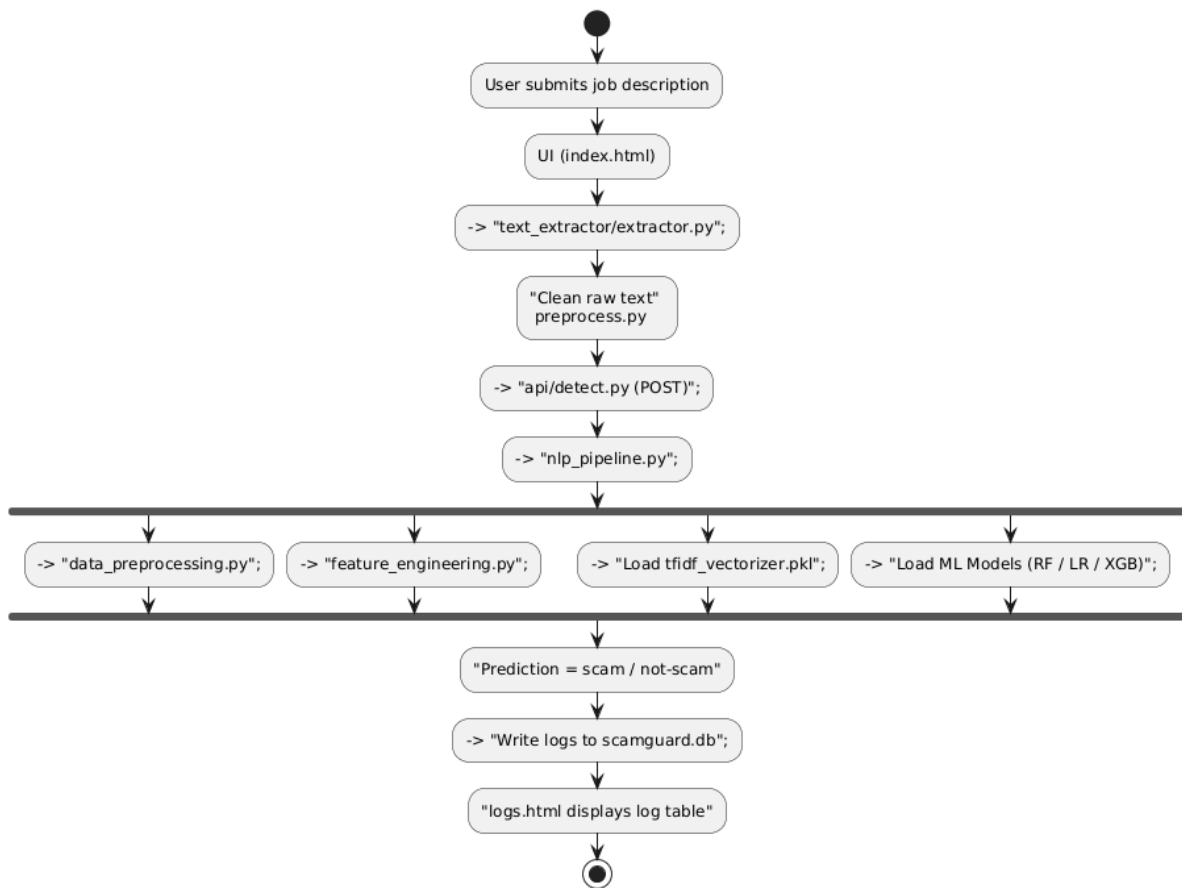
Asset Organization

- Data assets are stored under data/ and compressed in data.zip. This includes raw datasets, domain-lists (blacklist/whitelist), historical logs, or other resource files for scam detection or system initialization.
- Model artifacts are placed in models/. These represent persisted classifiers or data structures used to evaluate or flag scams.
- **cleaned_job_postings.csv**: The static data asset used for training.
- Frontend assets (HTML templates, CSS, JS, static images/icons) live in templates/ and static/, enabling separation of UI from backend logic.
- Configuration/Secrets (.env, credentials.json, token.json) are centralized to manage external integrations (e-mail, DB, external APIs) in a secure, maintainable way.

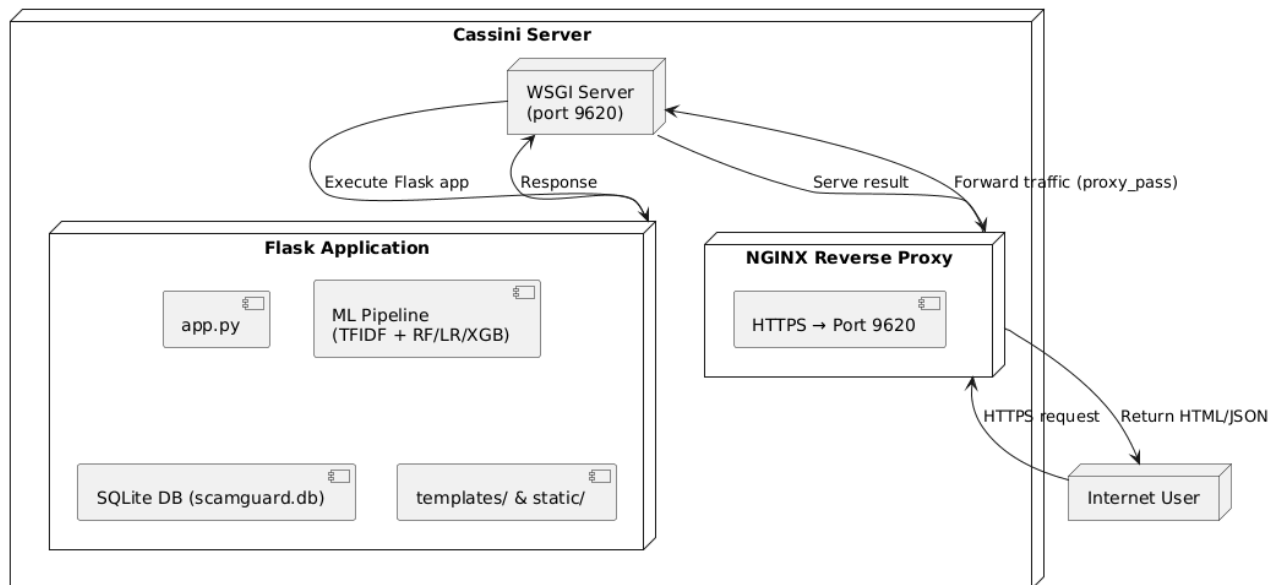
Module Interaction Diagram



ScamGuard - Full Data Flow Diagram



ScamGuard - Deployment Architecture (Cassini + Nginx)



Data and State Management

ScamGuard follows a stateless web-application architecture, meaning the server does not maintain long-term conversational data about users. Every prediction request is independent and processed in isolation. The application operates on a request-driven compute model:

1 request → 1 extraction + preprocessing pipeline → 1 prediction → response → state cleared.

The only long-lived state resides in in-memory machine-learning models and vectorizers loaded when the Flask application starts. All other computation occurs within the lifespan of a single HTTP request.

Application State

User State

- Although we have included authentication modules (auth.py, account.py), user login is only used for accessing admin/log pages. ScamGuard's primary prediction workflow remains stateless and does not require authentication.
- User-State Characteristics:
 - Flask sessions store minimal authentication info (e.g., session['user_id'])
 - Sessions are not used for prediction
 - No personal identifiers are stored
 - No JWTs or long-lived tokens are used
 - Prediction requests contain all necessary fields (title, description, etc.)
- Thus, for the main user workflow, each POST request is completely independent.

Application State

The application loads a small set of long-lived, read-only objects when the server starts. These objects remain in memory throughout the application lifetime and support efficient prediction:

Component	State Type	Purpose
TF-IDF Vectorizer (.pkl)	Immutable model state	Converts cleaned text → sparse vectors
Random Forest / XGBoost Models	Immutable ML parameters	Compute scam-risk probabilities
Feature Engineering Schema	Static configuration	Controls extraction of engineered features
SQLite Database Connection (db.py)	Persistent on-disk state	Store logs of prediction metadata

These follow a singleton lifetime model, loaded once, reused for all requests, ensuring fast inference and avoiding repeated disk I/O.

Data Persistence

ScamGuard uses a single persistent data store:

SQLite Database (scamguard.db)

Purpose:

- Log prediction requests
- Store extracted fields and metadata
- Track timestamps and model outputs
- Support admin monitoring and analytics

Characteristics:

- Lightweight and file-based
- No user profile or personal data stored
- Suitable for the application's scale and academic constraints

Data Loading

Data is sourced from three locations:

Local Application Assets (Loaded Once at Startup)

- tfidf_vectorizer.pkl
- scamguard_rf_model.pkl
- scamguard_xgb_model.pkl
- engineered_features.csv (feature schema)

These provide the foundational state for preprocessing and prediction for our application.

Runtime User Input (Loaded Per Request)

Each HTTP POST request supplies:

- Job title
- Job description
- Company name
- Location
- Additional context text

All transformations and computations occur in memory during that request only.

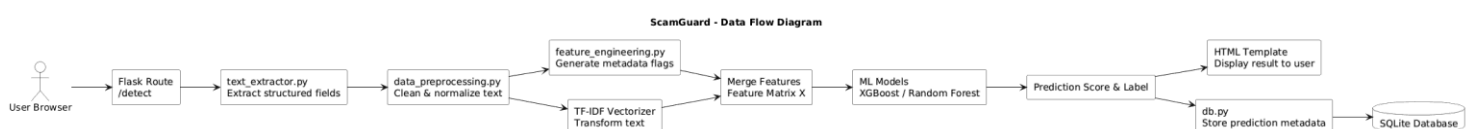
Database Read/Write Operations

ScamGuard performs:

- INSERT operations to log predictions
- SELECT operations for admin reporting

We haven't used any heavy file storage, cloud APIs, or external databases.

Data Flow for Prediction (Primary User Interaction)



Testing and Validation

Testing Approach We employed a multi-layered testing strategy to ensure robustness from the code level up to the user experience.

1. Unit & Integration Testing:

- **Data Pipeline:** We validated that the pipeline.py script could handle corrupt CSV rows without crashing. We specifically tested the on_bad_lines='skip' parameter against a test file containing malformed delimiters.
- **Preprocessing:** Validated that data_preprocessor.py correctly stripped HTML tags and identified stop words using NLTK.

2. Model Validation:

- We compared two architectures using an 80/20 train-test split.
- Baseline (Logistic Regression): Achieved 96% Recall.
- Champion (Random Forest): Achieved 97% Recall and 98.6% Accuracy.
- Validation Decision: We selected Random Forest because minimizing False Negatives (missed scams) is critical for user safety.

3. User Acceptance Testing (Manual Red Teaming):

- The team performed manual testing by sourcing real scam emails and legitimate job descriptions (from LinkedIn and Reddit) and pasting them into the live application on Cassini.
- Result: The system correctly flagged 9/10 known scams as "High Risk."

Results and Evaluation

Project Success:

The project was successful. We met our primary objective of deploying a functional, high-accuracy fraud detection tool accessible via a job description.

Key Metrics:

- **Recall (97%):** This was our "North Star" metric. It indicates that the system successfully catches 97% of fraudulent postings in the test set.
- **Accuracy (98.6%):** The overall correctness of the system is industry-standard.
- **Latency:** The application processes a request in under 500ms on the Cassini server.

Lessons Learned & Retrospective

- **Data Quality is Paramount:** We learned that real-world data is significantly "messier" than standard datasets. Dealing with *ParserError* and missing values took 40% of our development time.
- **Counter-Intuitive Insights:** Our assumption that "Remote Work = Scam" was statistically proven wrong. This taught us to trust data over intuition. In our dataset, remote jobs were safer, leading us to use it as a positive signal.
- **Deployment Complexity:** We underestimated the challenge of deploying large ML models. We learned the importance of Git LFS for handling binary artifacts in a CI/CD pipeline.

Future Work and Recommendations

While the current system is a robust MVP, we identified several areas for future expansion:

1. **Browser Extension:** Develop a Chrome/Edge extension that automatically scrapes job descriptions from sites like Indeed or LinkedIn as the user browses, injecting a "Safety Score" overlay directly onto the page.
2. **Gmail Integration:** Utilize the Gmail API to scan incoming job offer emails for phishing patterns, flagging them before the user even opens the message.
3. **Feedback Loop:** Implement a "Report Incorrect Result" button in the UI. This would allow us to collect "hard negatives" (scams the model missed) to retrain and improve the model over time, addressing the risk of Data Drift.
4. **Deep Learning:** With more powerful hardware (GPU support), we could explore Transformer-based models (BERT) to better understand context, though this would increase inference latency.

User Manual

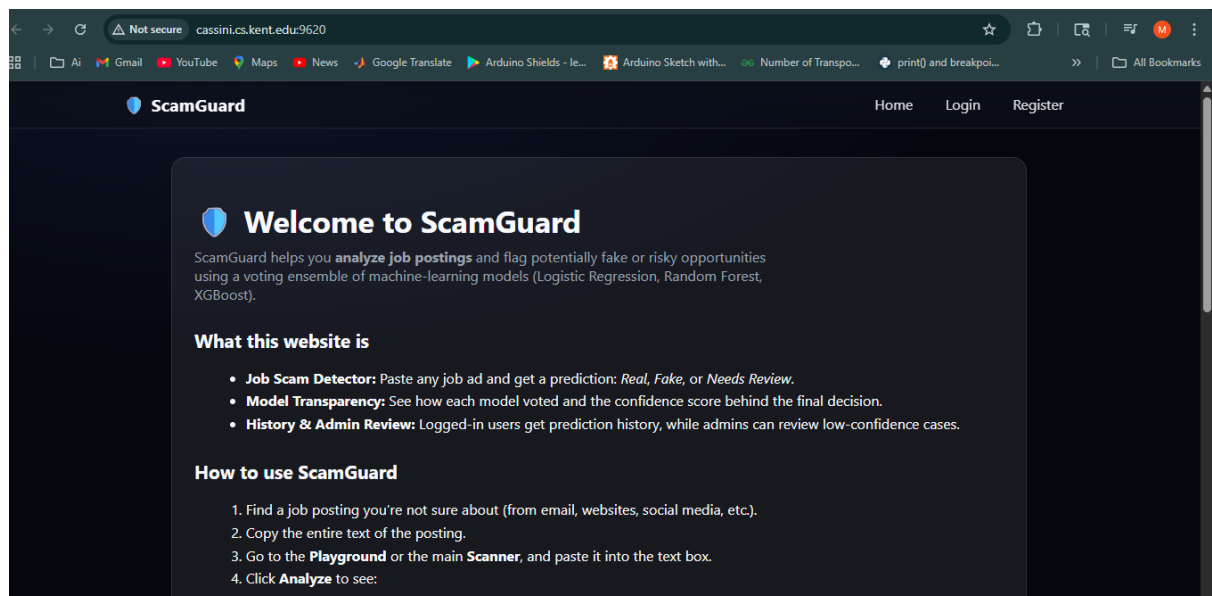
Accessing the Application

The production application is live and accessible at: URL: <http://cassini.cs.kent.edu:9620/>

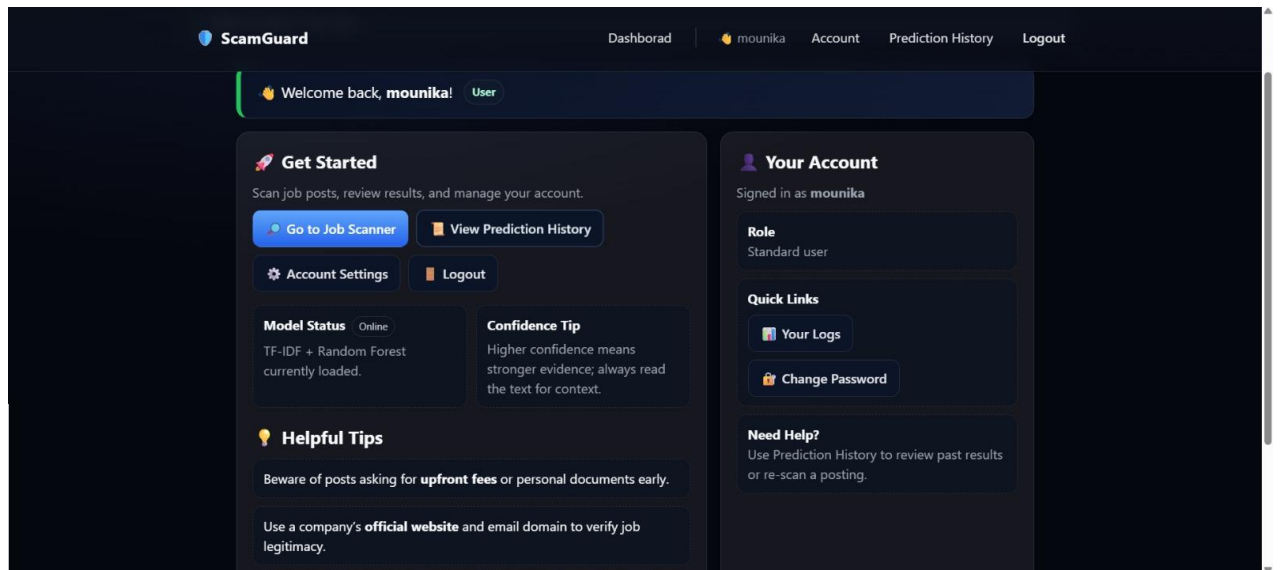
Installation (For Local Development) To run the application on your local machine:

1. **Clone the Repository**
git clone https://github.com/MounikaS26/ScamGuard.git
cd ScamGuard
2. **Pull Model Files (Git LFS):**
git lfs install
git lfs pull
3. **Install Dependencies:**
pip install -r requirements.txt
4. **Run the App:**
python app.py
5. **View:** Open your browser to <http://localhost:5000>.

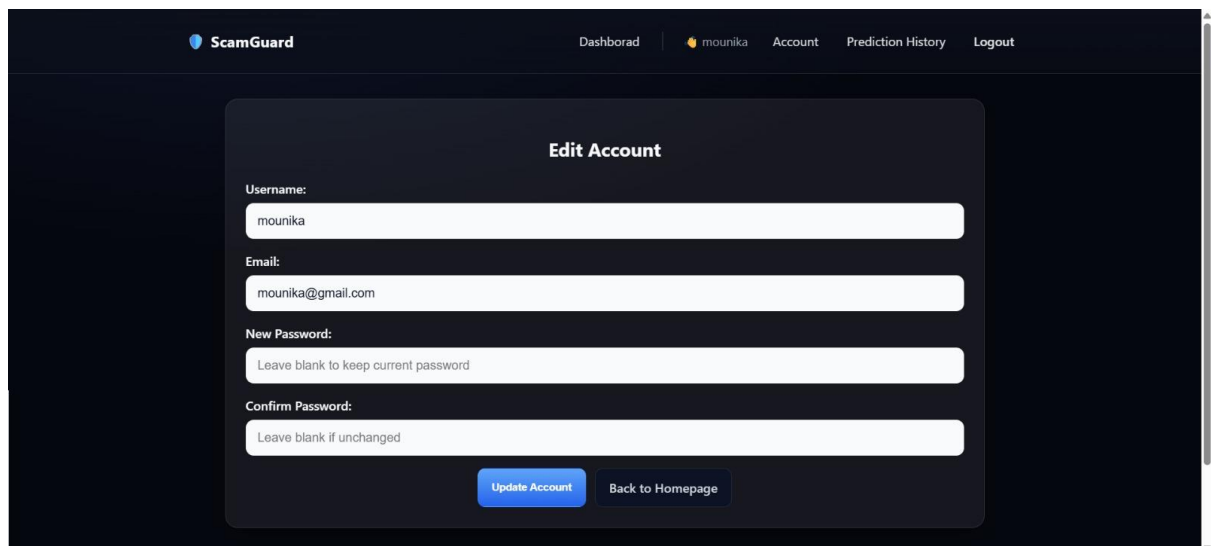
Home Page



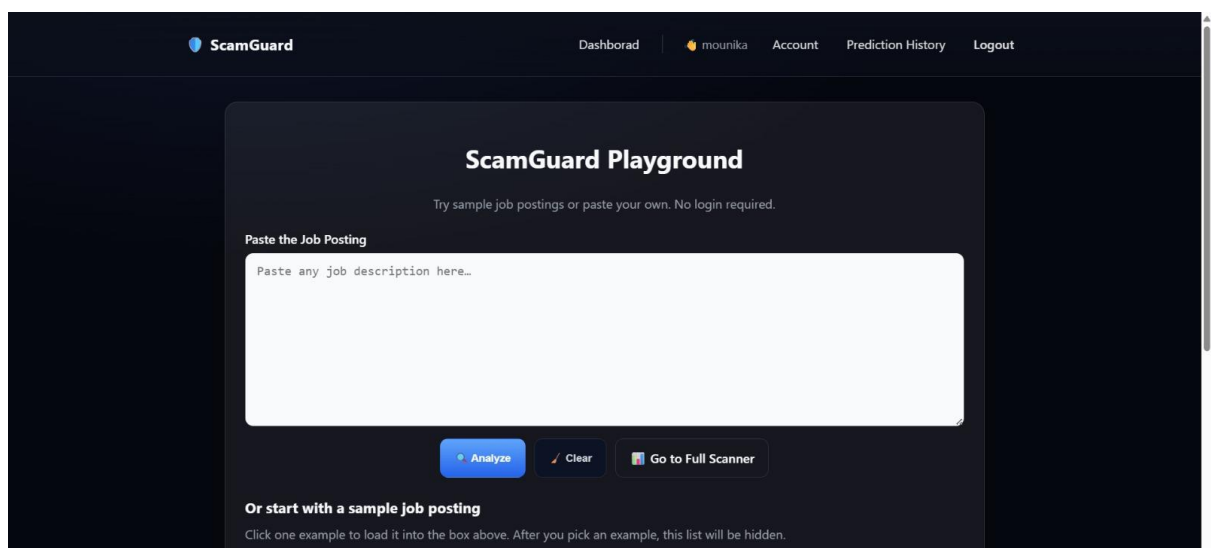
User Dashboard



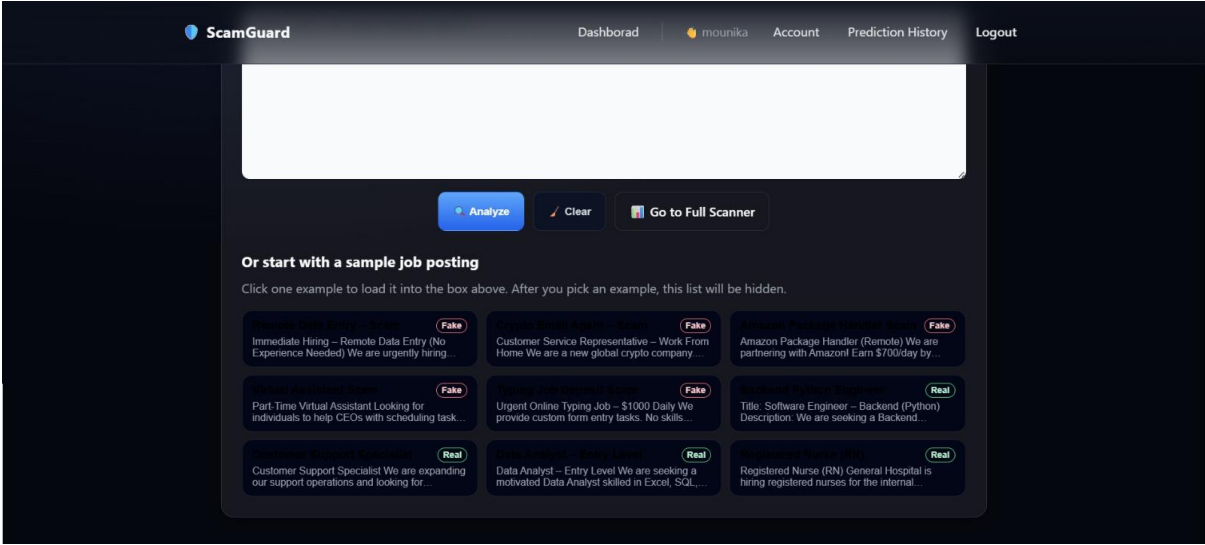
Account Settings



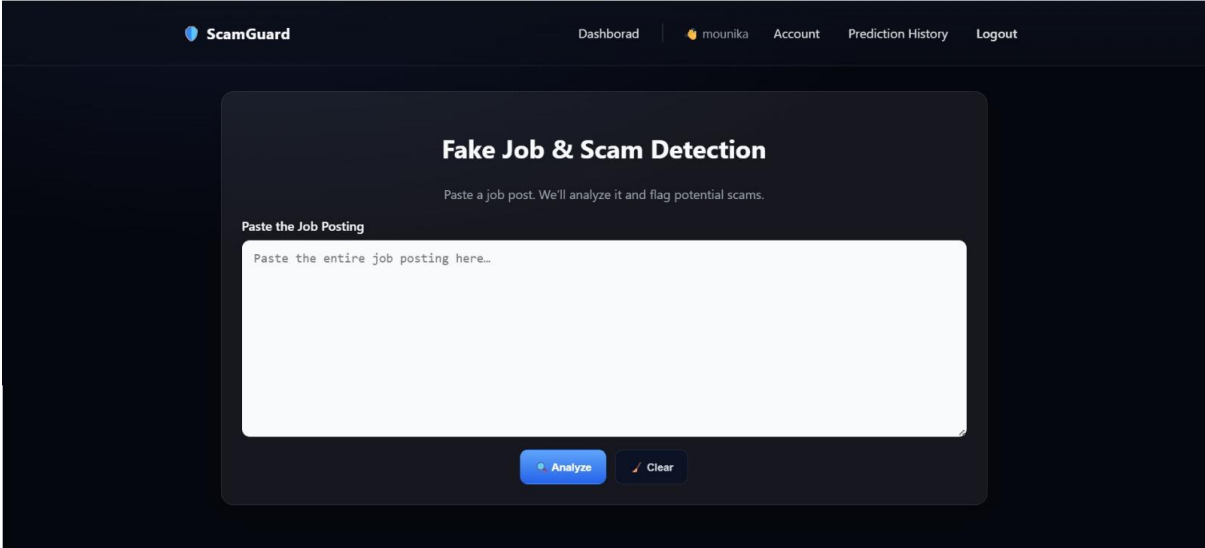
Playground



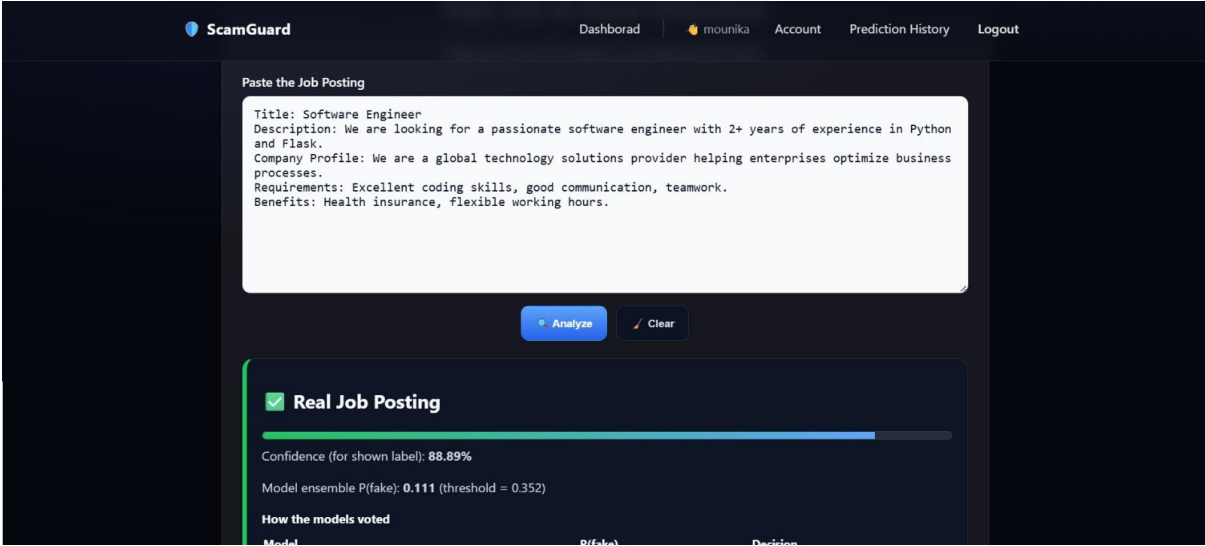
Sample Job Postings



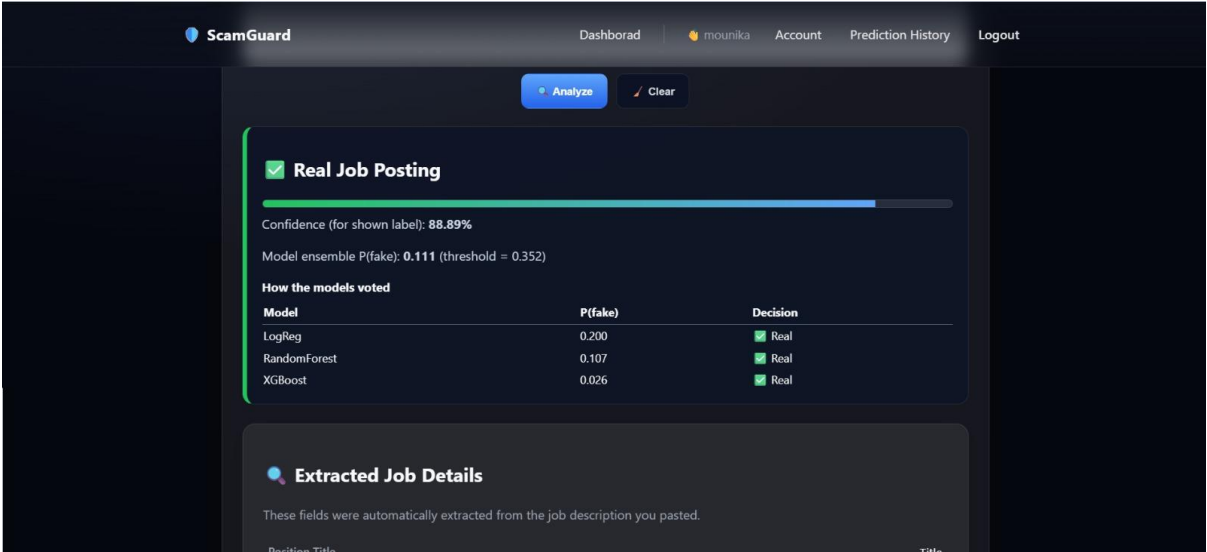
Initial Interface



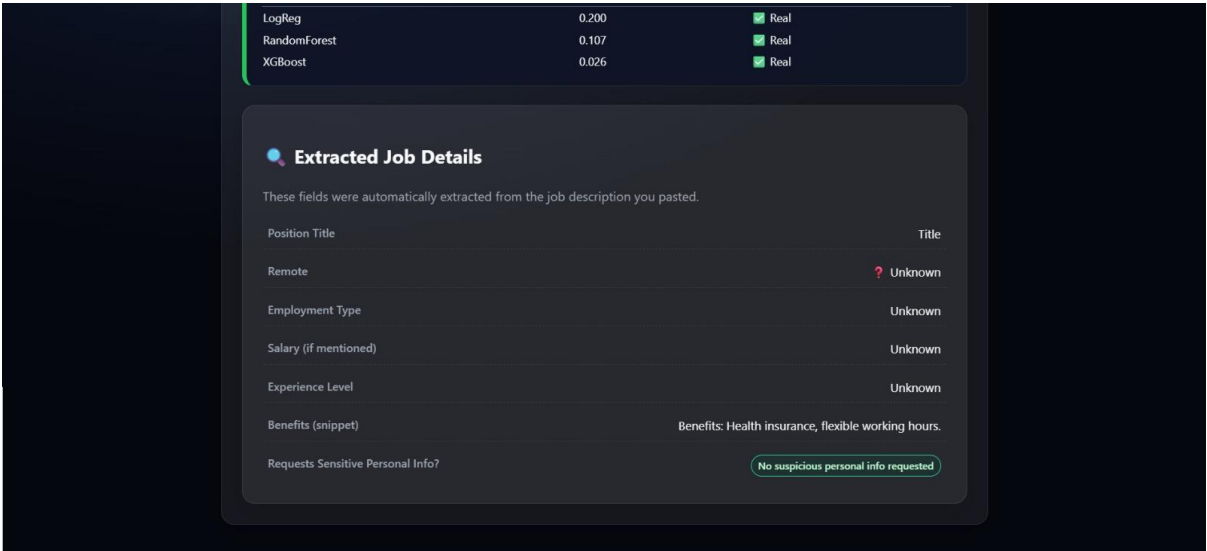
Real Job Example



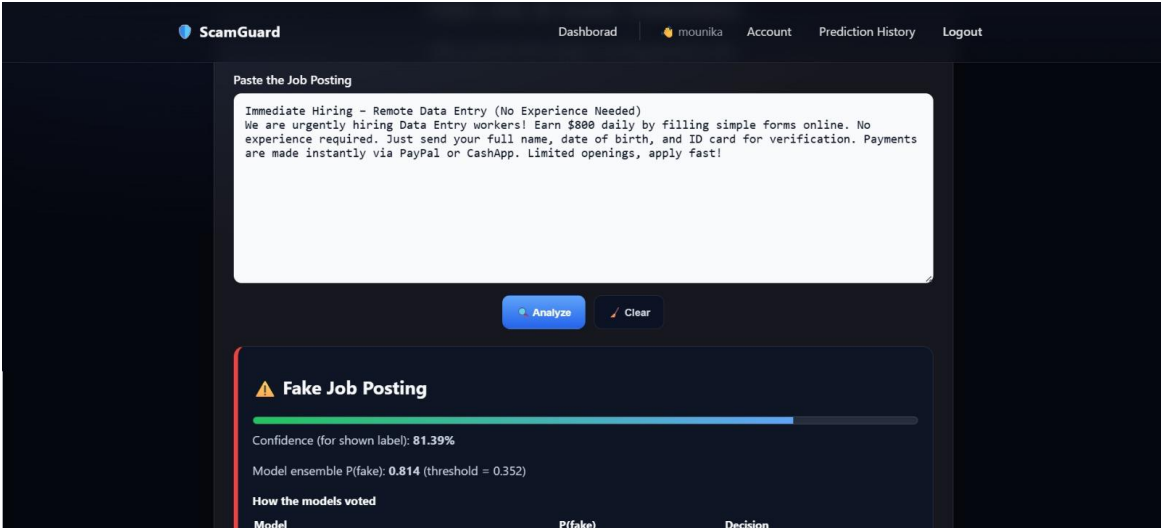
Real job Model Evaluation example



Job Details Extractor



Fake Job Posting



Fake Job Model Prediction

ScamGuard

Dashborad

mounika

Account

Prediction History

Logout

⚠ Fake Job Posting

Confidence (for shown label): 81.39%

Model ensemble P(fake): 0.814 (threshold = 0.352)

How the models voted

Model	P(fake)	Decision
LogReg	0.981	⚠ Fake
RandomForest	0.615	⚠ Fake
XGBoost	0.845	⚠ Fake

🔍 Extracted Job Details

These fields were automatically extracted from the job description you pasted.

Position Title

Data Entry

Remote

🌐 Remote

Job Details Extractor: Fake Job Posting

🔍 Extracted Job Details

These fields were automatically extracted from the job description you pasted.

Position Title

Data Entry

Remote

🌐 Remote

Employment Type

Unknown

Salary (if mentioned)

\$800 daily

Experience Level

No experience required

Benefits (snippet)

Not clearly mentioned

Requests Sensitive Personal Info?

⚠ Yes – Sensitive Info Requested

Needs Review: Form 1

ScamGuard

Dashborad

mounika

Account

Prediction History

Logout

Help us review this job posting

Our model marked this job as "Needs Review". Please answer a few short questions to help the review team decide whether it is real or a scam.

NEEDS REVIEW

Model decision: Needs Review

Confidence: 68.87%

Why we're asking

Some jobs fall into a grey area where the model is not confident enough to call them clearly **real** or clearly **fake**.

Your answers give the review team extra context that the text alone can't capture – especially things like **how** you received the job, whether money was requested, or if they asked for sensitive details.

1 Where did you find this job posting?

Example: "LinkedIn post", "WhatsApp message", "Unknown website", or paste the full URL.
e.g. LinkedIn – https://www.linkedin.com/jobs/...

Needs Review: Form 2

2

Did they ask you to pay any money, deposit, or fee?

Select...

If yes, please describe what kind of payment or fee they requested.

e.g. \$50 for training materials paid via gift cards...

3

Did they ask for sensitive personal information before an official offer?

Select...

Examples: ID card, SSN, bank details, selfie with ID, full home address, etc.

Describe anything they asked for that felt too personal...

4

Anything else that felt suspicious or unusual? (optional)

Short note for our review team – timing, behaviour, communication style, etc.

e.g. They refused video calls and pushed me to decide immediately...

Your responses are only used to improve review decisions for this job and similar cases.

Submit answers & return to scanner

Prediction History

Prediction History

[← Back to Prediction](#)

Search by title, text, label...

Tip: search by keywords or type **fake**, **real**, or **review** to narrow results.

All

Real

Fake

Needs Review

Job Title: Administrative Assistant

Needs Review

Reviewed

Company: Northview Services Company Overview: Northview Services is a small professional office providing general support functions for client-facing teams. Our internal structure is currently being updated, so a detaile...

View full text

Confidence

68.87%

#1

12/10/2025 22:19

Delete

Immediate Hiring – Remote Data Entry (No Experience Needed)

Fake

We are urgently hiring Data Entry workers! Earn \$800 daily by filling simple forms online. No experience required. Just send your full name, date of birth, and ID card for verification. Pay

View full text

Confidence

81.39%

#2

11/25/2025 00:48

Delete

Title: Software Engineer

Real

Description: We are looking for a passionate software engineer with 2+

Admin Dashboard

ScamGuard

Dashborad

venkatsai24

Account

Prediction History

Logout

Welcome back, venkatsai24!

Admin Dashboard

Manage users, monitor scans, and review suspicious predictions.

TOTAL SCANS

25

All job postings checked so far.

LABEL BREAKDOWN

Real: 0 Fake: 0 Needs Review: 0

How the model is classifying scanned jobs.

SCANS IN LAST 7 DAYS

0

Recent platform activity.

AVG. MODEL CONFIDENCE

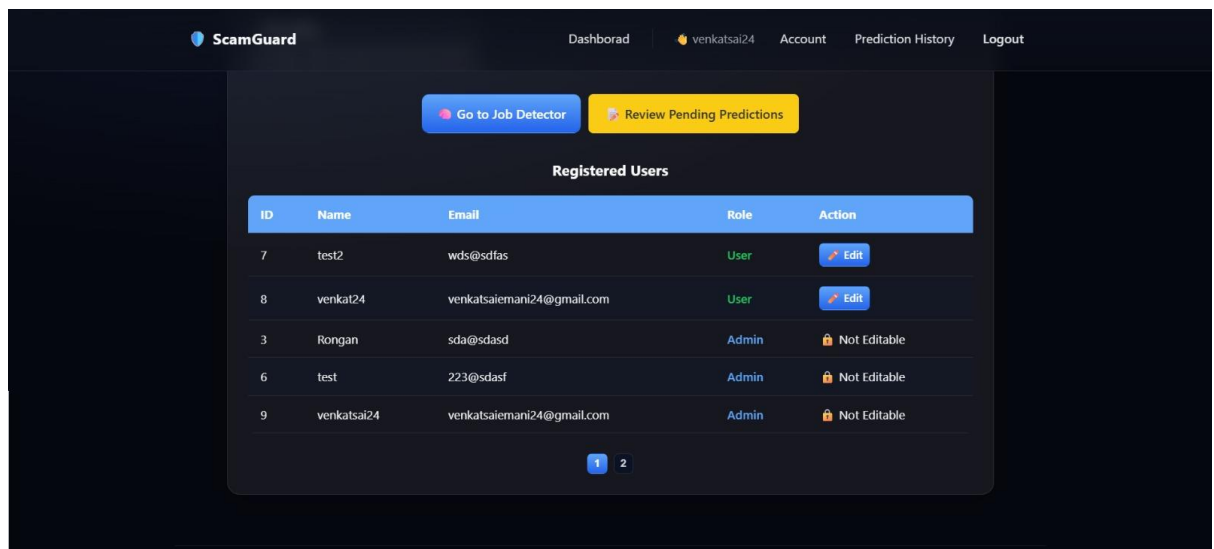
70.9%

Average confidence across stored predictions.

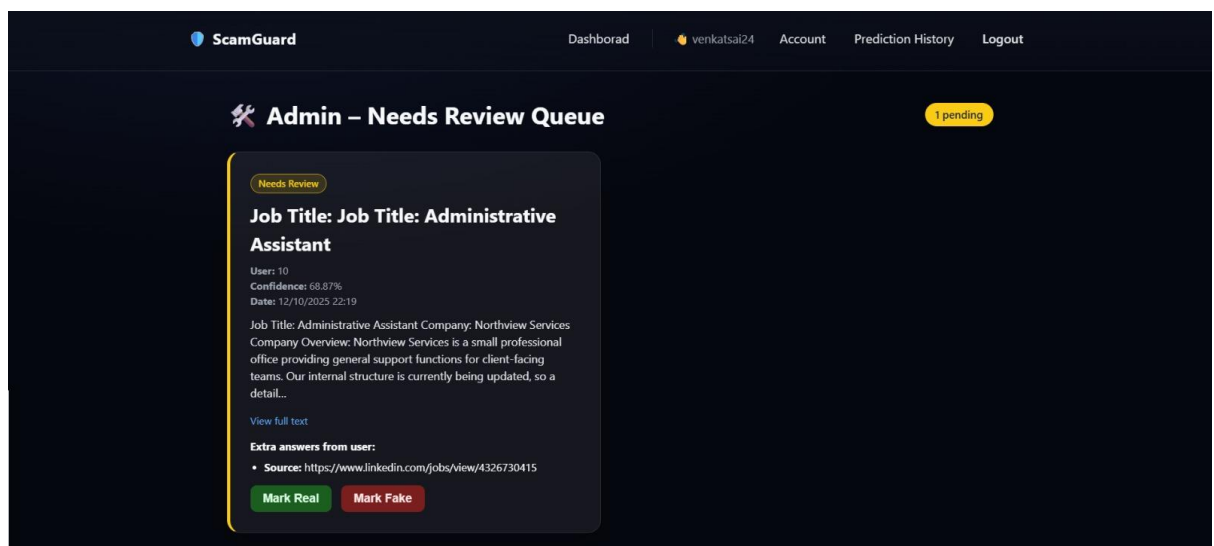
Go to Job Detector

Review Pending Predictions

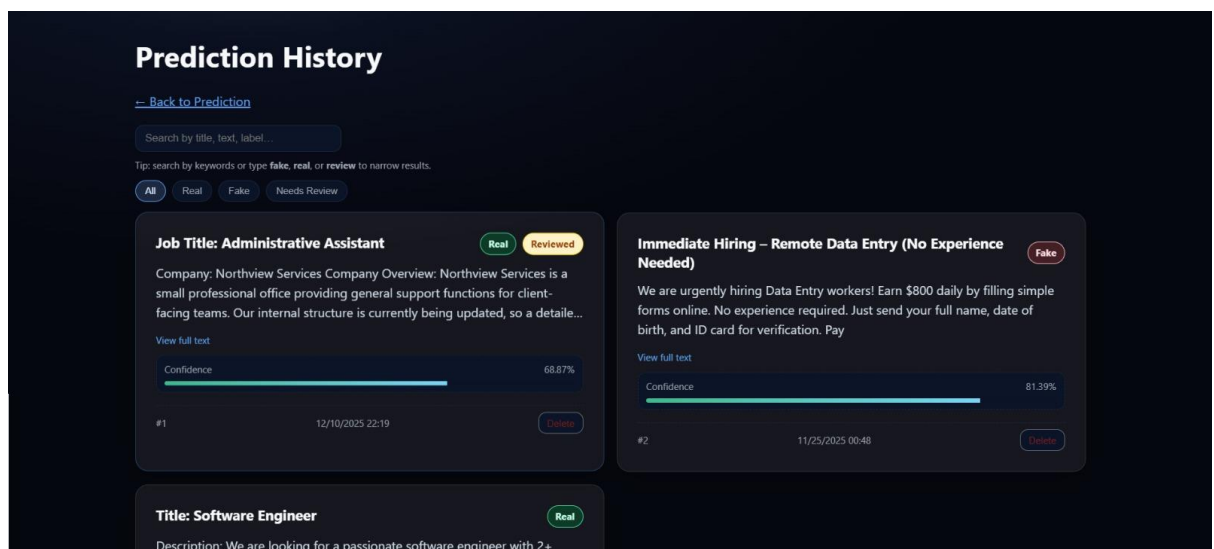
Admin Views



Admin Review Page



After Admin Prediction



Appendices

A. Project Resources

- GitHub Repository: <https://github.com/MounikaS26/ScamGuard>
- Live Deployment: <http://cassini.cs.kent.edu:9620/>

B. Sample Data Structure:

The model was trained on data following this schema (first 5 rows sample):

First 5 Rows:											
	title	location	company_name	company_profile	description	requirements	benefits	salary_range	employment_type	required_experience	required_education
0	Remote Virtual Assistant	Remote	AlphaWorks	NaN	Looking for self-starters. If you can start to...	No experience necessary. Training provided. Ag...	Health insurance, 401(k)	\$76250 per year	NaN	NaN	NaN
1	Data Entry Clerk	Delhi, India	Everest Group	NaN	We are looking for a motivated individual to j...	Must have basic computer skills. Reliable inte...	Flexible schedule	\$39690 per year	NaN	NaN	NaN
2	Remote Virtual Assistant	Houston, TX	Silverline	NaN	Looking for self-starters. If you can start to...	Must have basic computer skills. Reliable inte...	Work remotely, generous bonuses	\$70825 per year	NaN	NaN	NaN
3	Executive Assistant	Nairobi, Kenya	GreenTree Ltd	NaN	Exciting role with flexible hours. Training pr...	Must have basic computer skills. Reliable inte...	Work remotely, generous bonuses	\$488402 per year	NaN	NaN	NaN
	required_experience	required_education	industry	telecommuting	has_company_logo	has_questions	contact_email	url	fraudulent		
N	NaN	NaN	NaN	NaN	1.0	NaN	NaN	hr.alphaworks@gmail.com	http://alphaworks-hiring.com/apply	1	
N	NaN	NaN	NaN	NaN	0.0	NaN	NaN	recruit@everest-jobs.com	http://everest-hiring.com/apply	1	
N	NaN	NaN	NaN	NaN	1.0	NaN	NaN	jobs@silverline.com	http://silverline-hiring.com/apply	1	
N	NaN	NaN	NaN	NaN	0.0	NaN	NaN	jobs@greentree.careers	http://greentree-hiring.com/apply	1	